

AH Bot Plus V.2

Vendor-Guard Stack Quirk: Known Bug (Unpatched)

What the current code does, why it is wrong,
and what is staged for later

Repo: [tommyfrontino37/mod-ah-bot-plus](#) (master-baseline + test-staging @ 779e699)

Date: 2026-09-12

Status: BUG PRESENT - fix written and verified but deliberately NOT applied

Staged fix: [ahbot-vendorguard-stackfix.patch](#) (11 insertions, 4 deletions, 1 file)

Companion: [README_StackQuirk.md](#) (this folder) - the same story in markdown

Read this first

This document describes ACTUAL CURRENT behavior. Every example in it reproduces on the unpatched build: the vendor guard compares a stack's TOTAL price against a PER-UNIT cap, so fairly priced stacks of guarded items are refused whenever the total exceeds the single-unit cap. A fix exists, is verified, and sits staged in this folder - but the operator decision (2026-09-12) is to leave the code as-is for now. Section 4 lists what is staged so a future patch day is one command plus paperwork.

1. What the current code does (HEAD 1758-1769)

Buyer path, src/AuctionHouseBot.cpp, exactly as it stands today:

```
// Check that the item isn't listed above Vendor sell price. Out-of-range entries (items added after the
// price table was built) get UINT32_MAX, which behaves the same as "not sold by a vendor"
bool preventedOverpayingForVendorItem = false;
uint32 vendorSellPrice = prototype->ItemId < vendorItemsPrices.size()
    ? vendorItemsPrices[prototype->ItemId] : UINT32_MAX; // 1761, wrapped in print
if (PreventOverpayingForVendorItems && vendorSellPrice > 0)
{
    if (doBuyout && auction->buyout > vendorSellPrice) // 1764: TOTAL vs UNIT
    {
        doBuyout = false;
        preventedOverpayingForVendorItem = true;
    }
    if (doBid && calcBidAmount > vendorSellPrice) // 1769: TOTAL vs UNIT
    {
        doBid = false;
        preventedOverpayingForVendorItem = true;
    }
}
```

The implemented rule

```
auction->buyout / calcBidAmount .. TOTAL copper for the whole stack (unit price x N)
vendorSellPrice ..... PER-UNIT copper (item_template.SellPrice lookup)
Implemented rule ..... unit price <= cap / N (for a stack of N)
Intended rule ..... unit price <= cap (for any stack size)
```

Concretely: a 20-stack of Runic Healing Potions (cap 60s) is refused at any total above 60s - a per-potion ceiling of 3s. Twenty Lesser Healing Potions (cap 25c) are refused above 25c total. Singles behave normally: a Runic Healing Potion passes at 60s.

2. Why it is a defect, not a policy

The comment contradicts the code

Line 1758 says 'Check that the item isn't listed above Vendor sell price' - per-item language: one item's price against one cap. Lines 1764/1769 implement whole-stack language. The stated intent is the dimensionally coherent one; the implementation drifted from it.

The packaging paradox

Identical goods at an identical fair per-unit price get different verdicts depending only on bundling. Runic Healing Potion (33447), cap 60s, every unit at exactly 60s:

```
Single at 60s/unit ..... total 60s .... PASS
5-stack at 60s/unit ..... total 3g ..... REFUSE
20-stack at 60s/unit .... total 12g .... REFUSE
```

A rule where the box changes the verdict instead of the price is incoherent. 'Never pay above S/unit' is a policy; 'never pay above S total regardless of count' is a category error. The Sept 7 vendor-guard specification labels this comparison a quirk and prescribes the per-unit correction.

Direction of error: refusal only

The guard sits downstream of action selection and only flips doBuyout/doBid from true to false - it never sets a price or selects an action. So this bug can only convert 'would have bought' into 'refused'. No gold is at risk from it; the cost is quietly skipped legitimate stacks. That safe direction is why it survived: gold-losing bugs get caught in days.

3. Practical consequences (unpatched)

- Guarded stacks are effectively unbought: any stack whose total exceeds the single-unit cap is refused - i.e., nearly every fairly priced stack of a guarded item, across all 2,467 positive caps.
- Singles and non-stackables behave correctly and are unaffected.
- The seller never consults the guard: all 12,906 listing values are exactly what they would be with or without this bug.
- Flips still lose money: over-refusal blocks every profitable flip the correct rule would block, plus legitimate trade on top. Nothing about leaving this unpatched opens an exploit.
- Player-visible effect: fair stack listings of guarded goods sit ignored by the bot. This looks like ordinary thin trade, which is why nobody noticed.
- Documentation note: the README (Sec. 3-4) and three guide PDFs currently AGREE with the code - they describe whole-auction semantics. They are accurate TODAY and become stale only the day the patch lands. Do not 'fix' them before then.

4. Staged for later (not applied)

Fix: ahbot-vendorguard-stackfix.patch. One scaled comparison inside the existing gate (plus comment clarification and a DEBUG-only log line):

```
uint64 vendorStackCap = 0;
if (PreventOverpayingForVendorItems && vendorSellPrice > 0)
{
    vendorStackCap = uint64(vendorSellPrice) * pItem->GetCount();
    if (doBuyout && auction->buyout > vendorStackCap) ... refuse
    if (doBid && calcBidAmount > vendorStackCap) ..... refuse
}
```

- Semantics kept: strictly-greater-than (equal-to-cap passes), ceiling-not-price-setter, ordinary-budget test still applies on top. Singles bit-identical; uint64 math; pItem proven safe at this site.
- Verification already done: byte-identical independent regeneration, clean apply-check on plain HEAD, braces 0/0, stub syntax check with zero warnings (see VERIFICATION.md).
- To apply later: git checkout test-staging && git apply --check <patch> && git apply <patch>, rebuild, restart.
- Paperwork staged with it: README_stackquirk_snippet.md (3 README paste ops) and Section 5 of AHBot_VendorGuard_StackQuirk_ErrorAnalysis.pdf (every stale statement in the README + 3 PDFs with before/after text). Apply the paperwork the same day the patch lands.
- Live test for patch day: Runic 20-stack at 60s each (12g, cap 60s/unit) - refused now, bought after; same stack at 1g each (20g) - refused now and after.